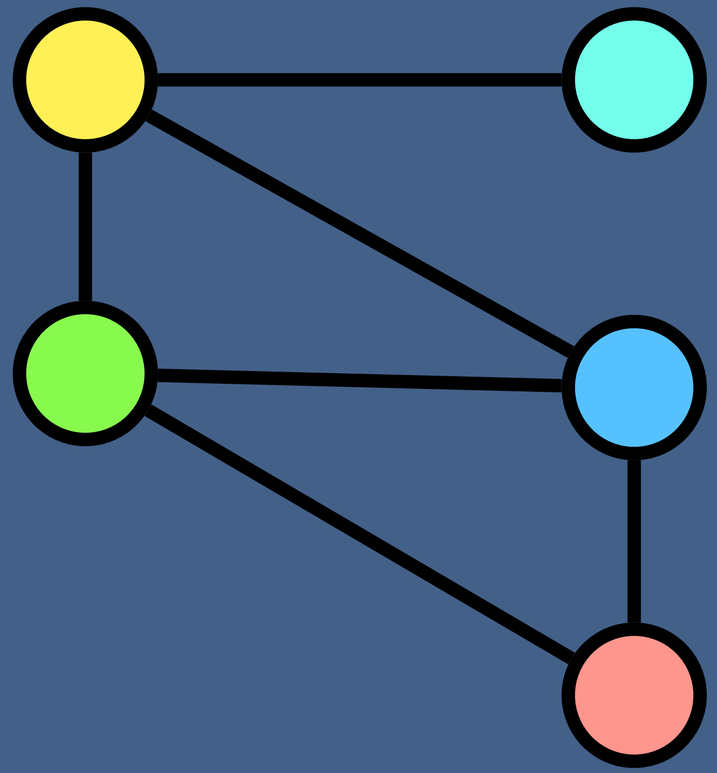
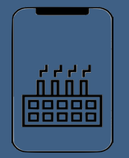
Graph



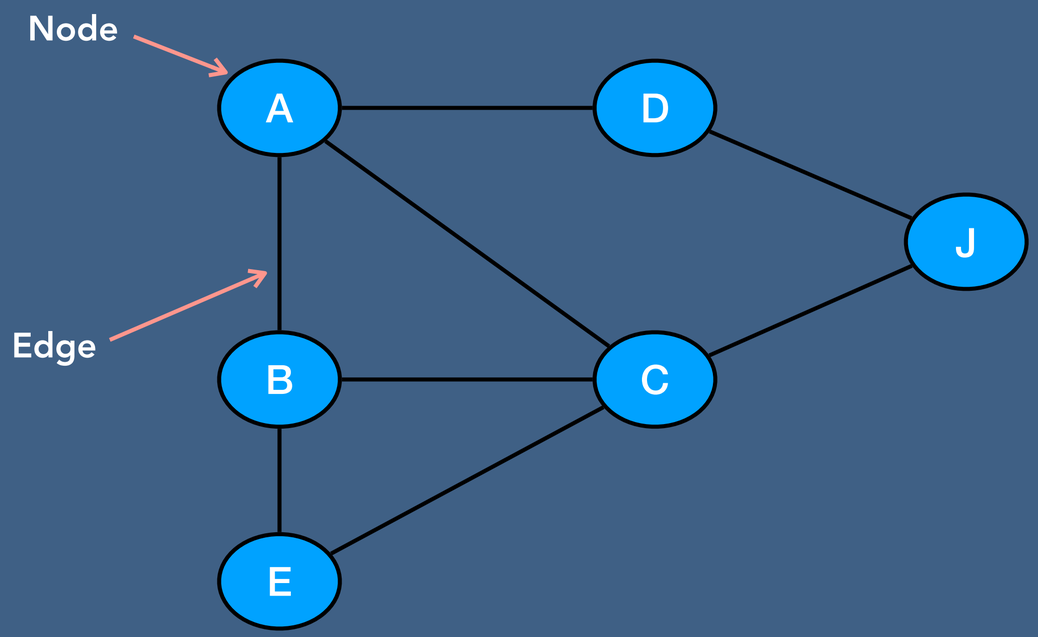
What you will learn

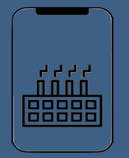
- What is a graph? Why do we need it?
- Graph Terminologies
- Types of graphs. Graph Representation
- Traversal of graphs. (BFS and DFS)
- Topological Sorting
- Single source shortest path (BFS, Dijkstra and Bellman Ford)
- All pairs shortest path (BFS, Dijkstra, Bellman Ford and Floyd Warshall algorithms)
- Minimum Spanning Tree (Kruskal and Prim algorithms)



What is a Graph?

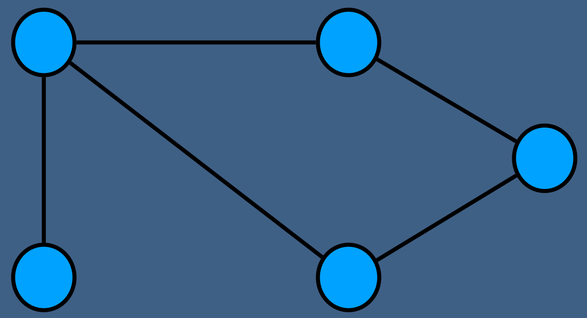
Graph consists of a finite set of Vertices(or nodes) and a set of Edges which connect a pair of nodes.

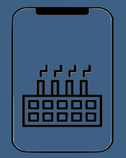




Why Graph?

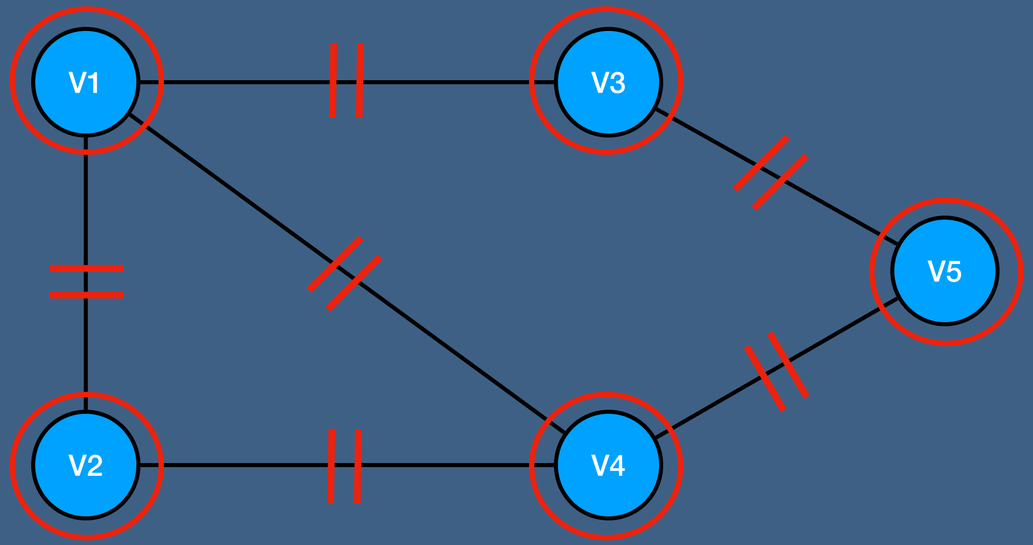

North Sea
Edinburgh
United Kingdom
Manchester
Liverpool
London
Amsterdam
Netherlands
Brussels
Belgium
Luxembourg
Paris
France
Switzerland
Monaco
Andorra
Barcelona
Madrid
Valencia
Spain
Granada
Málaga
Algiers
Tunis
Malta
Tyrrhenian Sea
Milan
Italy
Rome
Denmark
Hamburg
Berlin
Germany
Cologne
Frankfurt
Munich
Austria
Vienna
Slovakia
Budapest
Hungary
Czechia
Prague
Poland
Warsaw
Lithuania
Vilnius
Minsk
Belarus
Latvia
Moscow
Москва
Нижний Новгород
Kazan
Samara
Voronezh
Saratov
Volgograd
Kyiv
Київ
Kharkiv
Ukraine
Rostov-on-Don
Moldova
Chișinău
Odesa
Krasnodar
Slovenia
Zagreb
Croatia
Belgrade
Bosnia and Herzegovina
Serbia
Sarajevo
Romania
Bucharest
Sofia
Bulgaria
Black Sea
Podgorica
Kosovo
North Macedonia
Tirana
Albania
Greece
Athens
Istanbul
Bursa
Ankara
Turkey
İzmir
Antalya
Adana
Cyprus
Syria
Georgia
Tbilisi
Makhachkala
Caspian Sea
Armenia
Azerbaijan
Baku
Tehran





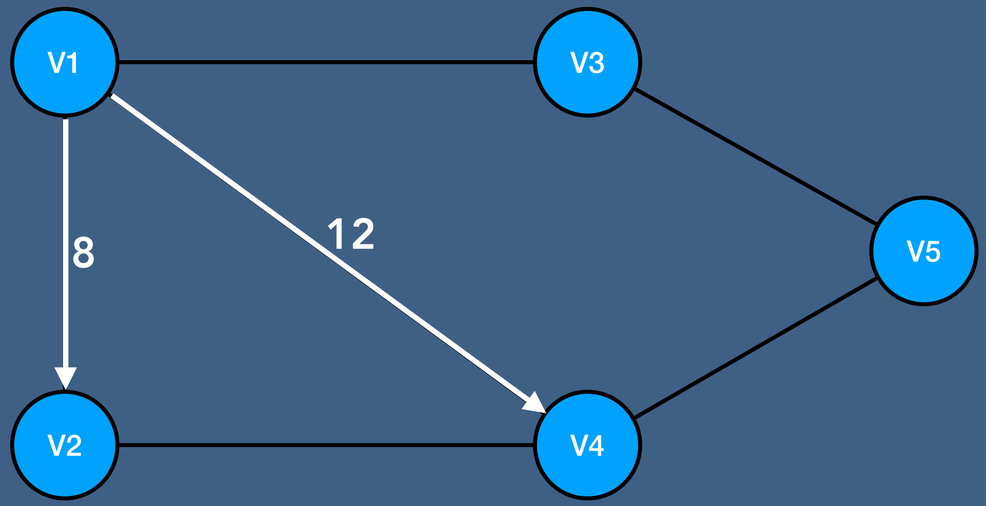
Graph Terminology

- **Vertices (vertex)** : Vertices are the nodes of the graph
- **Edge** : The edge is the line that connects pairs of vertices



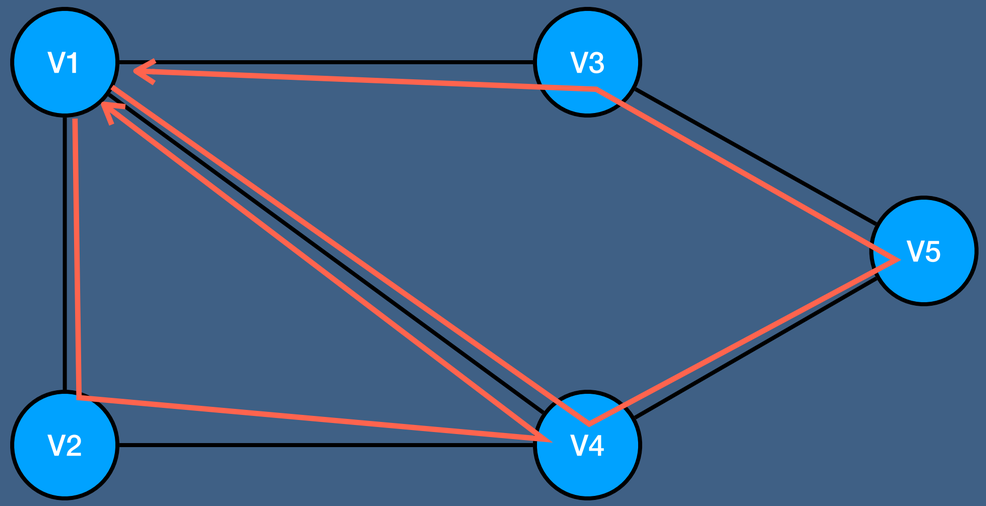
Graph Terminology

- **Vertices** : Vertices are the nodes of the graph
- **Edge** : The edge is the line that connects pairs of vertices
- **Unweighted graph** : A graph which does not have a weight associated with any edge
- **Weighted graph** : A graph which has a weight associated with any edge
- **Undirected graph** : In case the edges of the graph do not have a direction associated with them
- **Directed graph** : If the edges in a graph have a direction associated with them



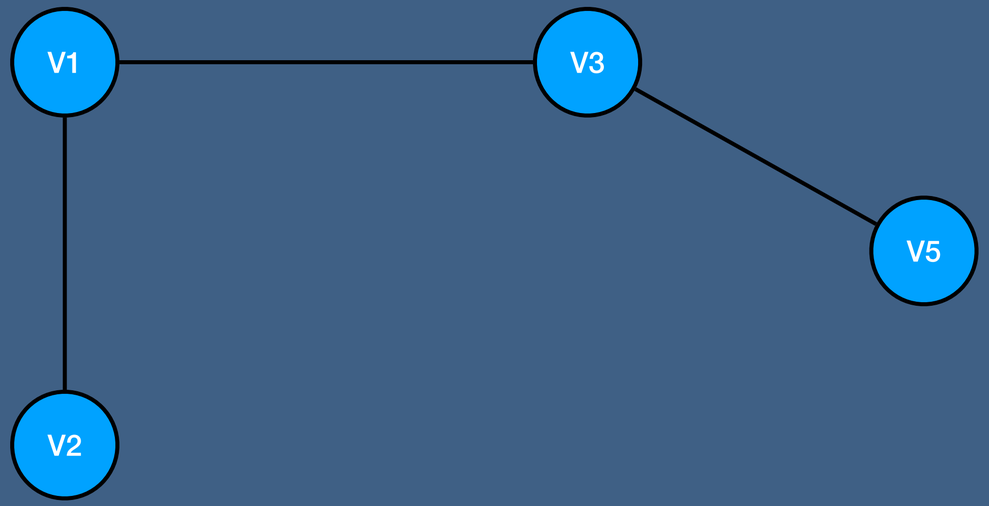
Graph Terminology

- **Vertices** : Vertices are the nodes of the graph
- **Edge** : The edge is the line that connects pairs of vertices
- **Unweighted graph** : A graph which does not have a weight associated with any edge
- **Weighted graph** : A graph which has a weight associated with any edge
- **Undirected graph** : In case the edges of the graph do not have a direction associated with them
- **Directed graph** : If the edges in a graph have a direction associated with them
- **Cyclic graph** : A graph which has at least one loop



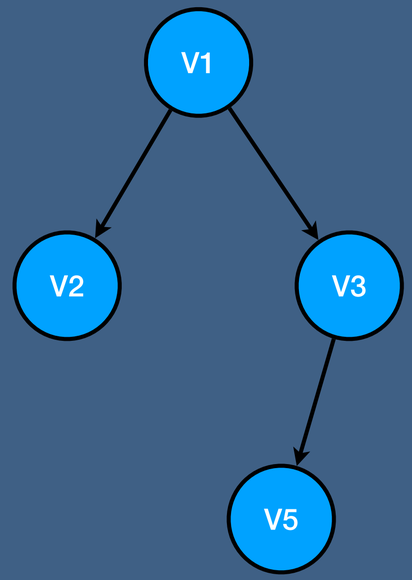
Graph Terminology

- **Vertices** : Vertices are the nodes of the graph
- **Edge** : The edge is the line that connects pairs of vertices
- **Unweighted graph** : A graph which does not have a weight associated with any edge
- **Weighted graph** : A graph which has a weight associated with any edge
- **Undirected graph** : In case the edges of the graph do not have a direction associated with them
- **Directed graph** : If the edges in a graph have a direction associated with them
- **Cyclic graph** : A graph which has at least one loop
- **Acyclic graph** : A graph with no loop

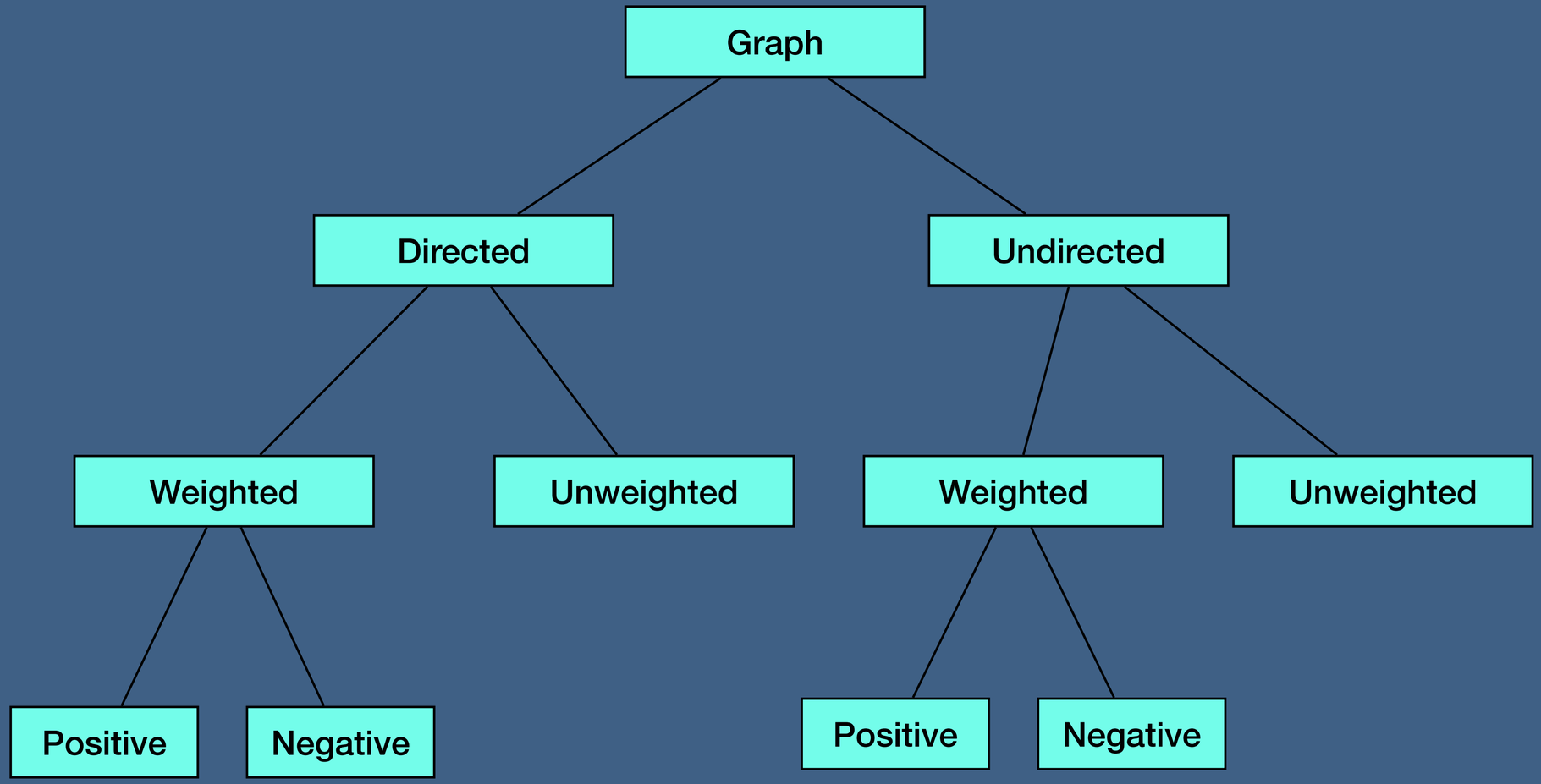


Graph Terminology

- **Vertices** : Vertices are the nodes of the graph
- **Edge** : The edge is the line that connects pairs of vertices
- **Unweighted graph** : A graph which does not have a weight associated with any edge
- **Weighted graph** : A graph which has a weight associated with any edge
- **Undirected graph** : In case the edges of the graph do not have a direction associated with them
- **Directed graph** : If the edges in a graph have a direction associated with them
- **Cyclic graph** : A graph which has at least one loop
- **Acyclic graph** : A graph with no loop
- **Tree**: It is a special case of directed acyclic graphs

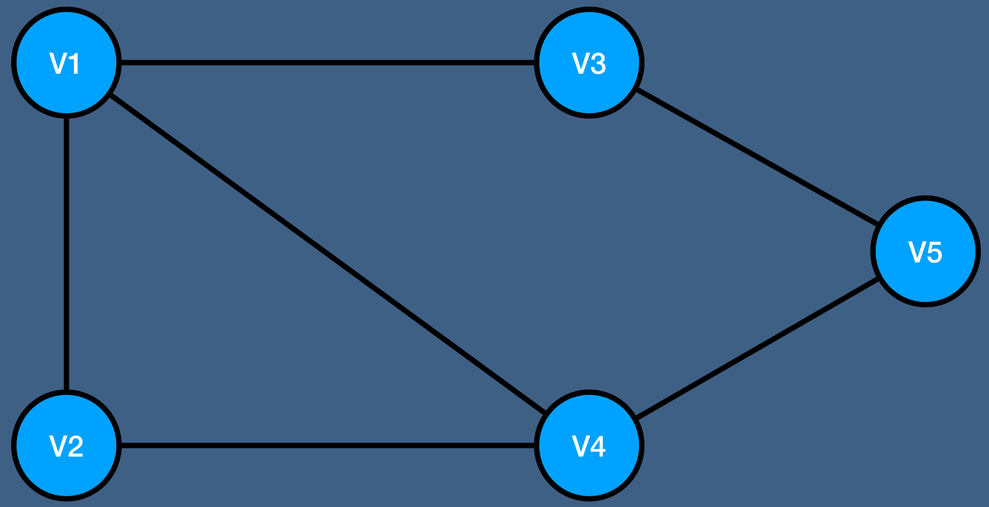


Graph Types


Graph
Directed
Undirected
Weighted
Unweighted
Weighted
Unweighted
Positive
Negative
Positive
Negative

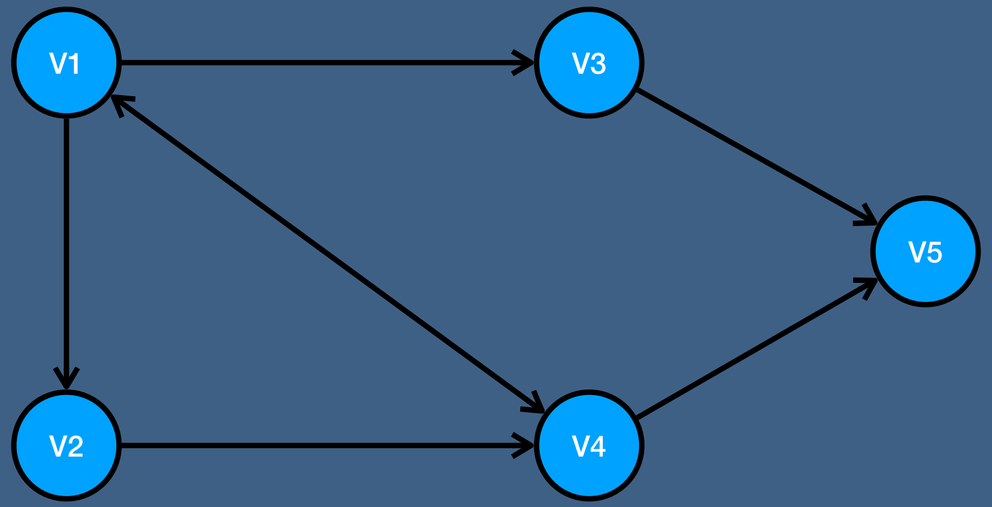
Graph Types

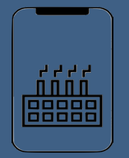
1. Unweighted - undirected



Graph Types

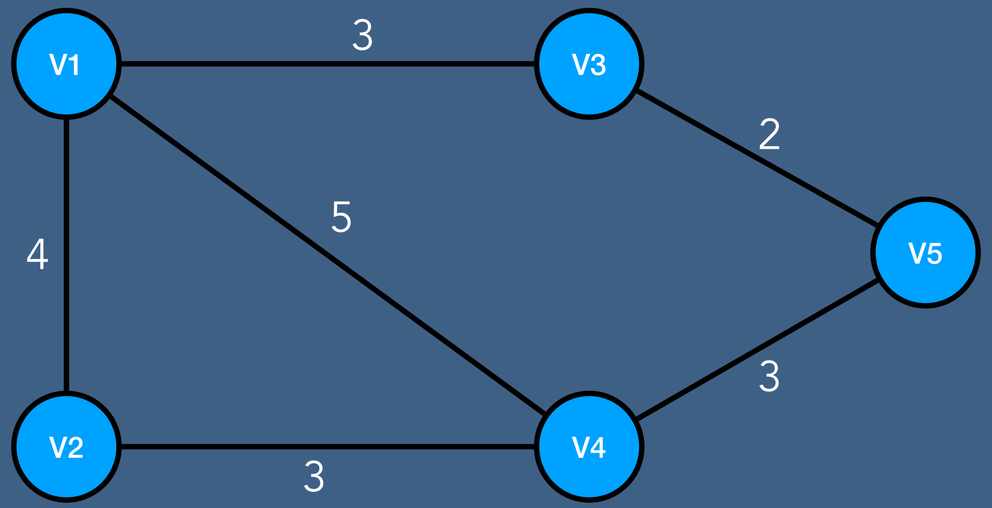
1. Unweighted - undirected
2. Unweighted - directed

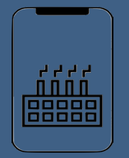




Graph Types

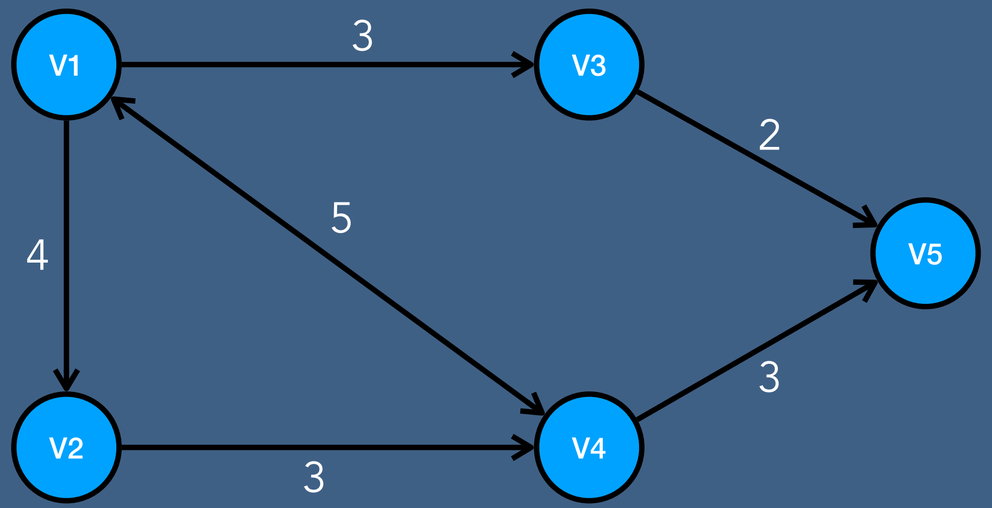
1. Unweighted - undirected
2. Unweighted - directed
3. Positive - weighted - undirected

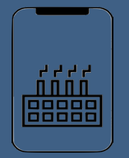




Graph Types

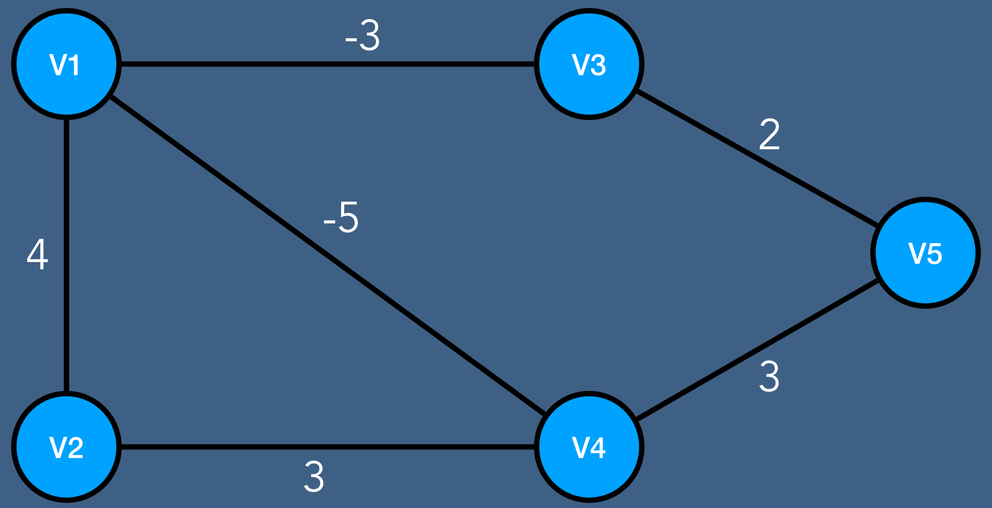
1. Unweighted - undirected
2. Unweighted - directed
3. Positive - weighted - undirected
4. Positive - weighted - directed

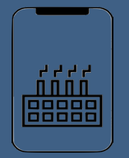




Graph Types

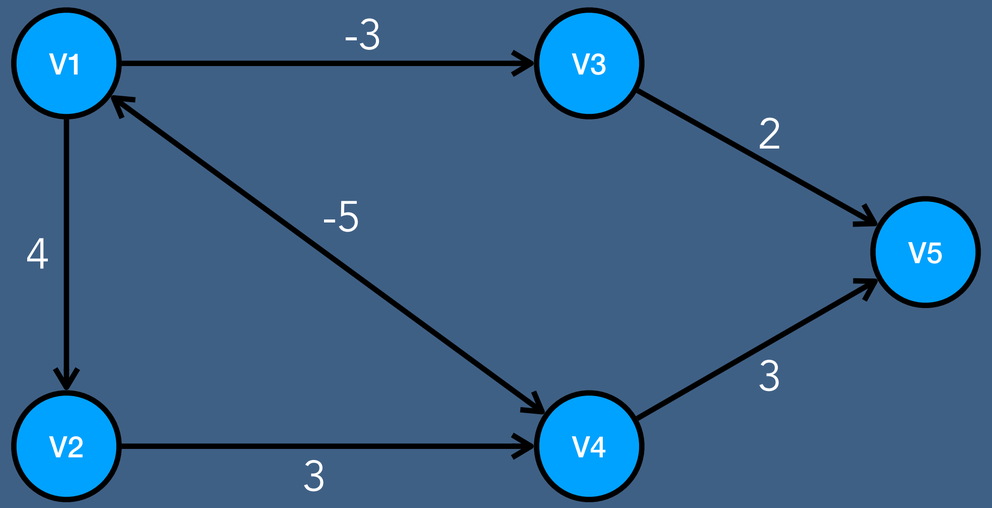
1. Unweighted - undirected
2. Unweighted - directed
3. Positive - weighted - undirected
4. Positive - weighted - directed
5. Negative - weighted - undirected





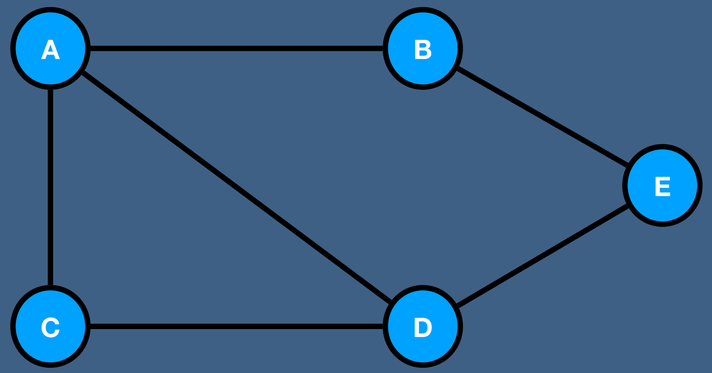
Graph Types

1. Unweighted - undirected
2. Unweighted - directed
3. Positive - weighted - undirected
4. Positive - weighted - directed
5. Negative - weighted - undirected
6. Negative - weighted - directed

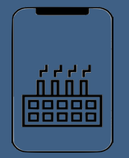


Graph Representation

Adjacency Matrix : an adjacency matrix is a square matrix or you can say it is a 2D array. And the elements of the matrix indicate whether pairs of vertices are adjacent or not in the graph

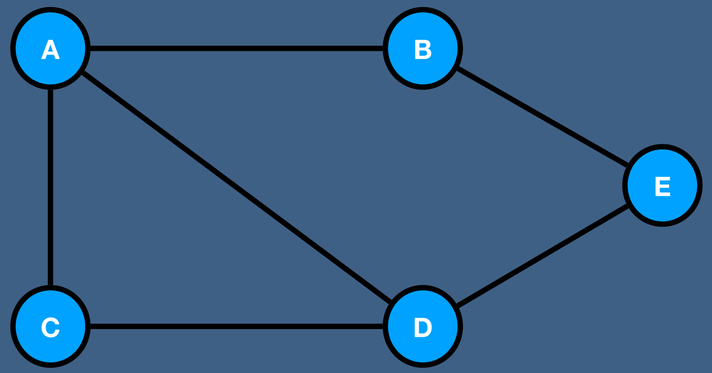


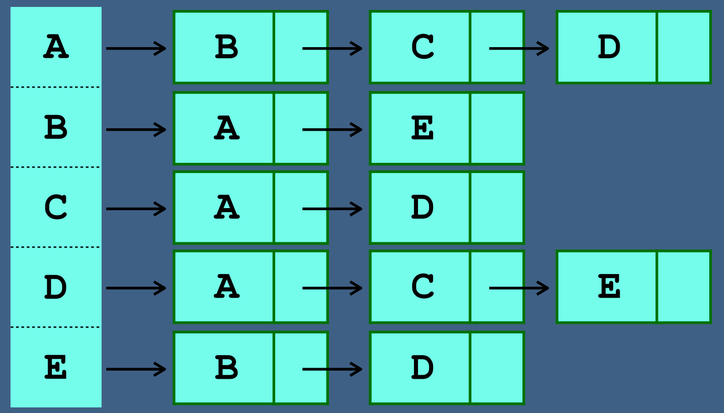
	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	0	1
C	1	0	0	1	0
D	1	0	1	0	1
E	0	1	0	1	0

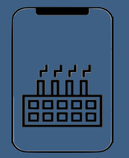


Graph Representation

Adjacency List : an adjacency list is a collection of unordered list used to represent a graph. Each list describes the set of neighbors of a vertex in the graph.





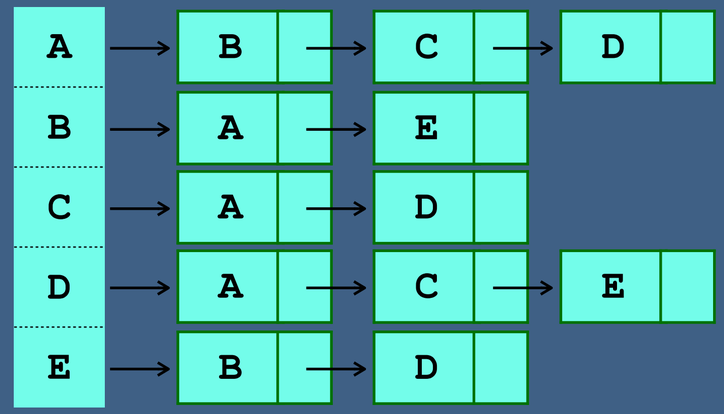


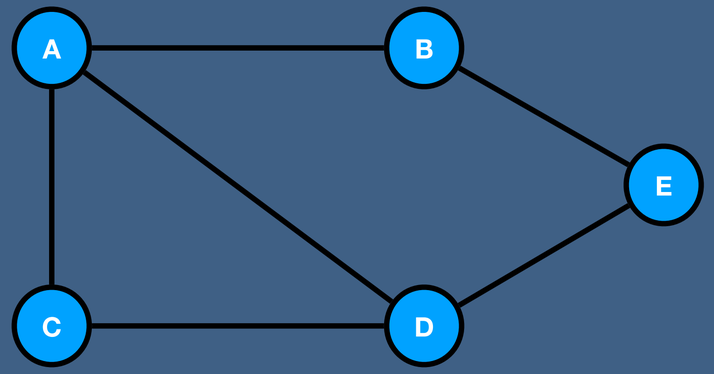
Graph Representation

If a graph is complete or almost complete we should use **Adjacency Matrix**

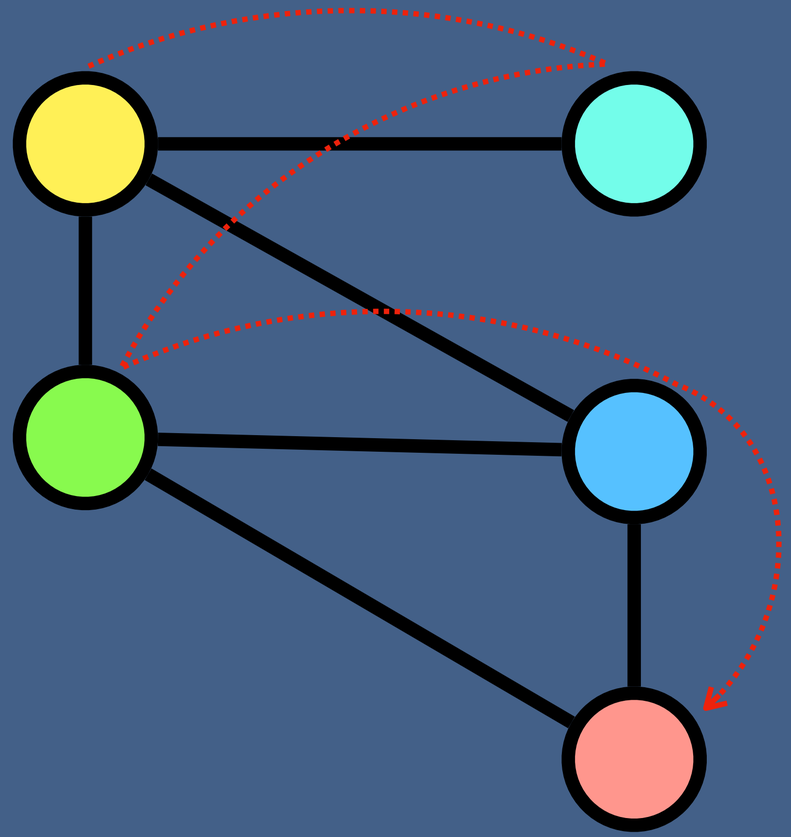
If the number of edges are few then we should use **Adjacency List**

	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	0	1
C	1	0	0	1	0
D	1	0	1	0	1
E	0	1	0	1	0



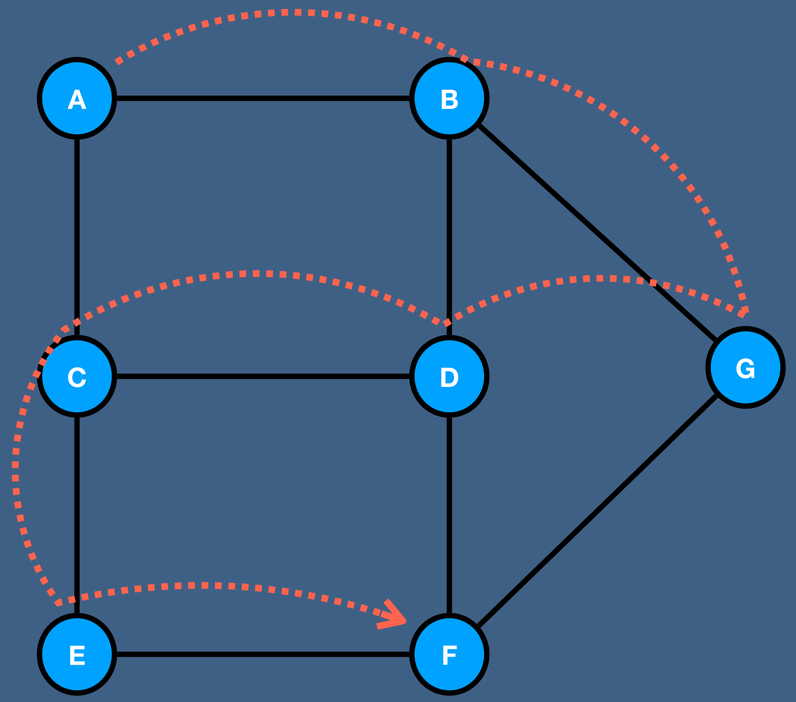


Graph Traversal

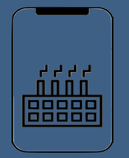


Graph Traversal

It is a process of visiting all vertices in a given Graph

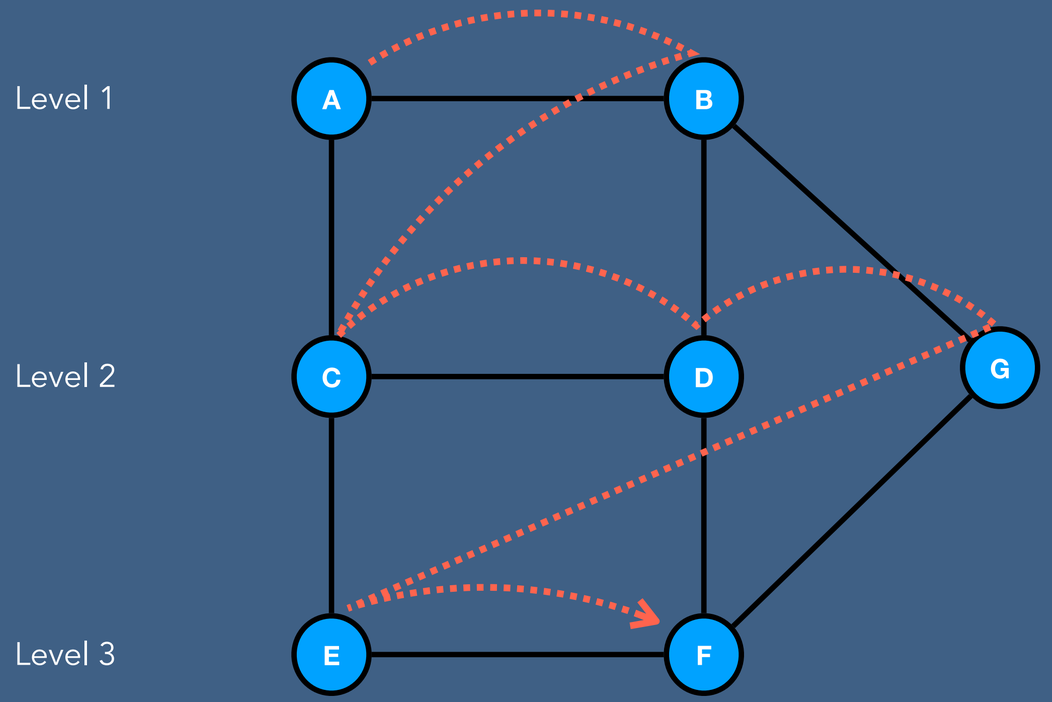


- Breadth First Search
- Depth First Search



Breadth First Search (BFS)

BFS is an algorithm for traversing Graph data structure. It starts at some arbitrary node of a graph and explores the neighbor nodes (which are at current level) first, before moving to the next level neighbors.

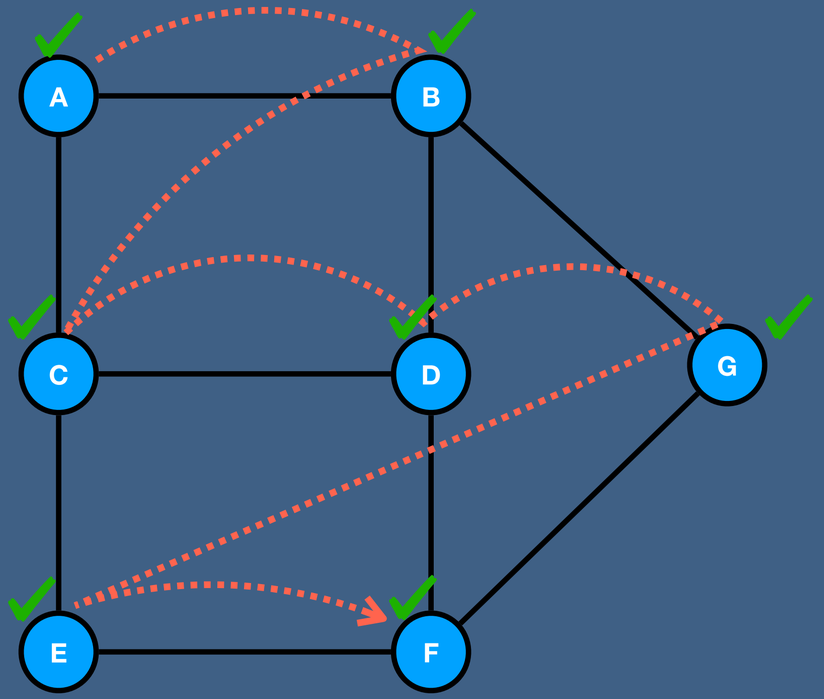


Breadth First Search Algorithm

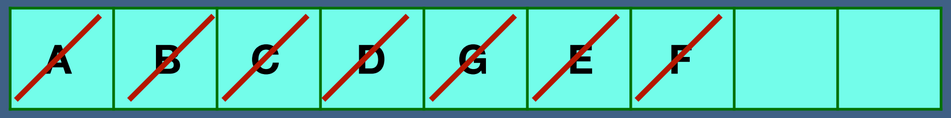
BFS

```
enqueue any starting vertex
while queue is not empty
  p = dequeue()
  if p is unvisited
     mark it visited
     enqueue all adjacent
     unvisited vertices of p
```

A B C D G E F



Queue



Time and Space Complexity of BFS

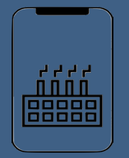
```
BFS

while all vertices are not explored ..... O(V)
  enqueue any starting vertex ..... O(1)
  while queue is not empty ..... O(V)
   p = dequeue() ..... O(1)
   if p is unvisited ..... O(1)
     mark it visited ..... O(1)
     enqueue unvisited adjacent vertices of p ..... O(Adj)
```

O(Adj) + O(E)

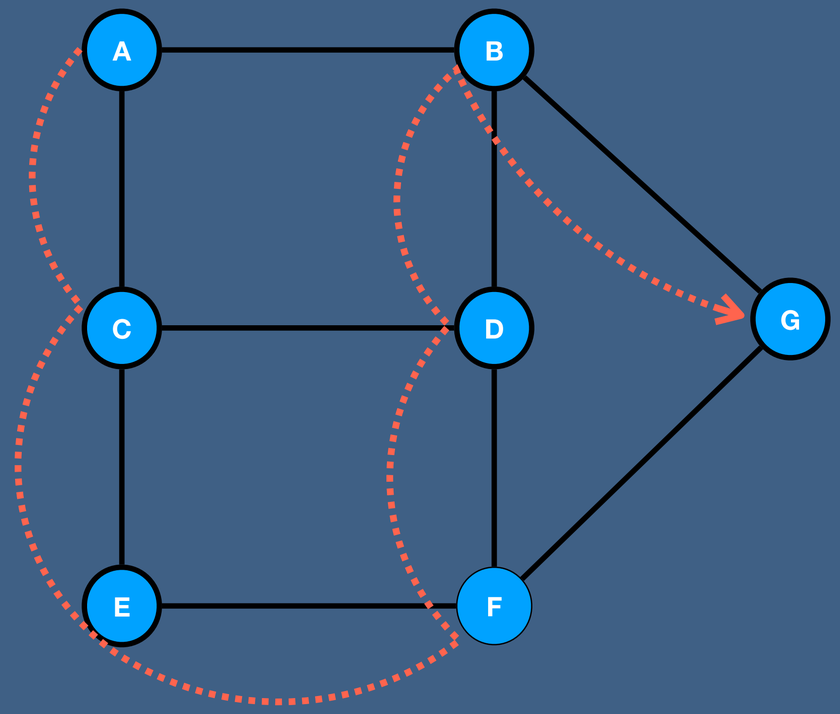
Time Complexity : O(V+E)

Space Complexity : O(V+E)



Depth First Search (DFS)

DFS is an algorithm for traversing a graph data structure which starts selecting some arbitrary node and explores as far as possible along each edge before backtracking



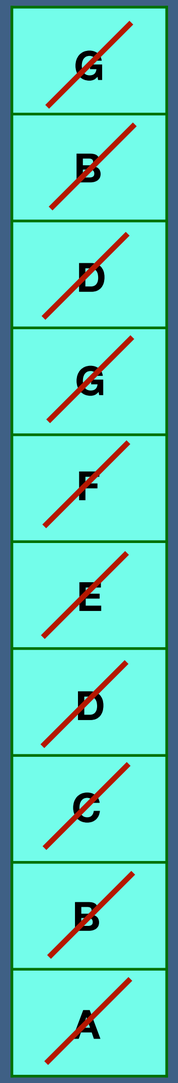
Depth First Search Algorithm

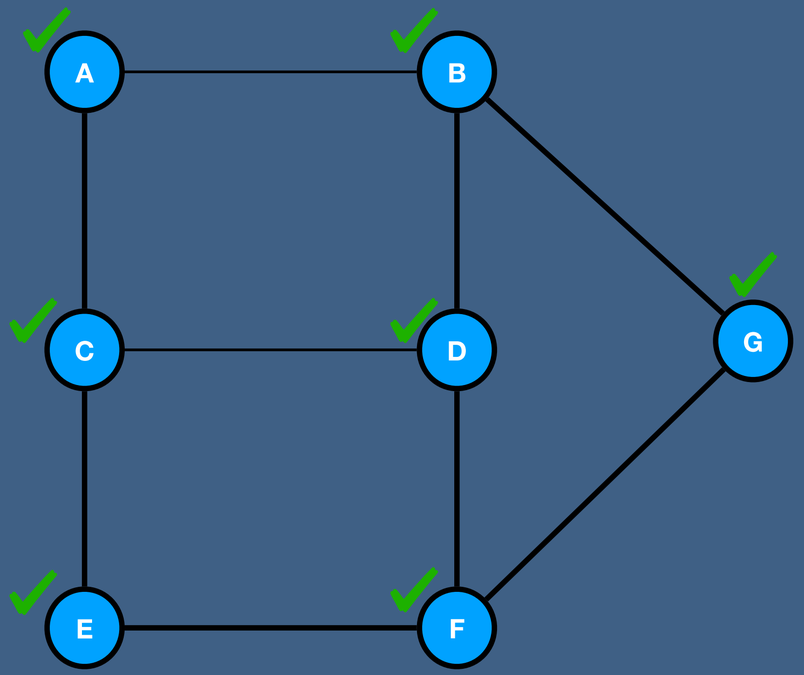
DFS

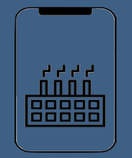
```
push any starting vertex
while stack is not empty
  p = pop()
  if p is unvisited
     mark it visited
     Push all adjacent
     unvisited vertices of p
```

A C E F D B G

Stack





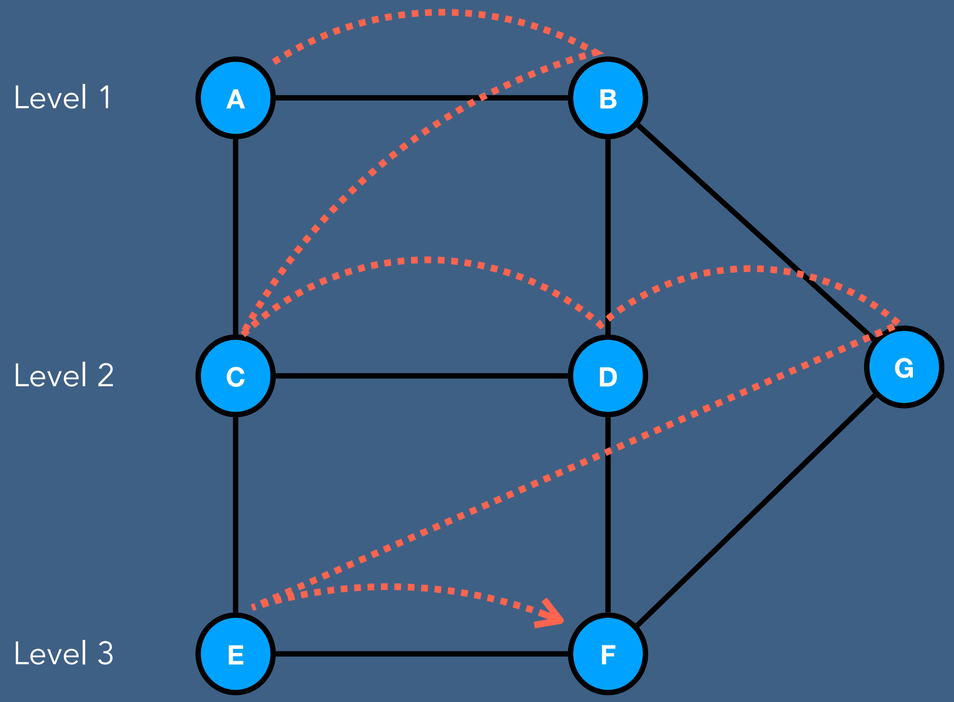


Time and Space Complexity of DFS

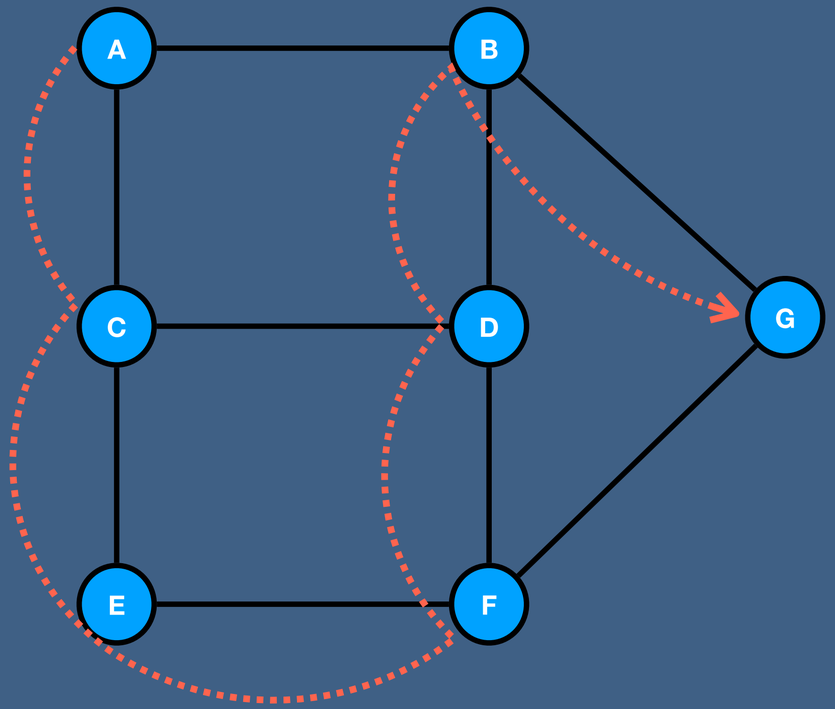
Time Complexity : O(V+E)

Space Complexity : O(V+E)

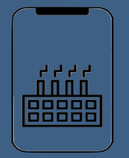
BFS vs DFS



BFS



DFS



BFS vs DFS

	BFS	DFS
How does it work internally?	It goes in breath first	It goes in depth first
Which DS does it use internally?	Queue	Stack
Time Complexity	O(V+E)	O(V+E)
Space Complexity	O(V+E)	O(V+E)
When to use?	If we know that the target is close to the staring point	If we already know that the target vertex is buried very deep